

Introduction to Data Systems and NoSQL

Abu Raihan Mostofa Kamal

Professor, CSE Department
Islamic University of Technology (IUT)
Gazipur-1704 Bangladesh
 raihan.kamal@iut-dhaka.edu
 +8801843925543

August 12, 2026

Chapter Outline

Reliable, Scalable, and Maintainable Applications

Motivation Behind NoSQL

Concept of Distributed Databases

ACID and BASE

Consistency

Aggregation

Key Value Database

Graph Database



Data Intensive Applications

- In today's word, many applications are **data-intensive**, as opposed to compute-intensive.
- The larger problem is the size or amount of data, the complexity of data and speed at which it changes or updates.
- A data-intensive application is typically built from standard building blocks that provide **commonly needed functionality**.

Example:

- ▶ **Store data** so that they, or another application, can find it again later (databases)
- ▶ **Remember the result** of an expensive operation, to speed up reads (caches)
- ▶ Allow users to **search data** by keyword or filter it in various ways (search indexes)
- ▶ Send a **message to another process**, to be handled asynchronously (stream processing)
- ▶ **Periodically crunch** a large amount of accumulated data (batch processing)

There are many database systems with **different characteristics**, because different applications have different requirements. And it can be hard to combine tools when you need to do something that a **single tool cannot do alone**.

Database Applications to Data Systems : Modern Trend

- In early days, applications were purely database query driven such as Payroll Management System in early 90's.
- Now in recent times, applications have been **diversified** and **diversified features** are added on one application to meet the requirements of various stakeholders. Such as traditional operation of HR management, sms service, real-time tracking, environment monitoring and so on.
- Hence, database applications now converses to **Data Systems**.
- Many new tools for data storage and processing have emerged in recent years.
One technology does not fit all requirements. **For example**, there are datastores that are also used as message queues (Redis), and there are message queues with database-like durability guarantees (Apache Kafka)



Database Applications to Data Systems : Modern Trend (Cont.)

In designing modern data systems a major concern is: **"How does the system provide consistently good performance to clients?"**

- **Reliability:** The system should continue to work correctly (performing the correct function at the desired level of performance) even in the face of adversity (hardware or software faults, and even human error).
- **Scalability:** As the system grows (in data volume, traffic volume, or complexity), there should be reasonable ways of dealing with that growth.
- **Maintainability:** Over time, many different people will work on the system (engineering and operations, both maintaining current behavior and adapting the system to new use cases).

We will explain Scalability a bit more.



Data Systems : Scalability

Scale of a System

Scalability is the term we use to describe a systems ability to cope with increased load. It is very application specific. But in general it deals with the following basic questions:

1. If the system grows in a particular way, what are our options for coping with the growth?
2. How can we add computing resources to handle the **additional load**?

Load of a System

Load

Load can be described with a few numbers which we call **load parameters**. The best choice of parameters depends on the architecture of your system. **Example:**

- Requests per second to a web server
- The ratio of reads to writes in a database
- The number of simultaneously active users in a chat room
- The hit rate on a cache

Load of a System: Example

System Load: Twitter (X)

Lets consider Twitter (now X) as an example, using data published in November 2012. Two of Twitters main operations are:

1. **Post tweet** A user can publish a new message to their followers (4.6k requests/sec on average, over 12k requests/sec at peak).
2. **Home timeline** A user can view tweets posted by the people they follow (300k requests/sec).

Challenge:

- Simply handling 12,000 writes per second (the peak rate for posting tweets) would be fairly easy.
- However, Twitters scaling challenge is not primarily due to tweet volume, but due to **fan-out** i.e. each user follows many people, and each user is followed by many people.

Load of a System: Example (Cont.)

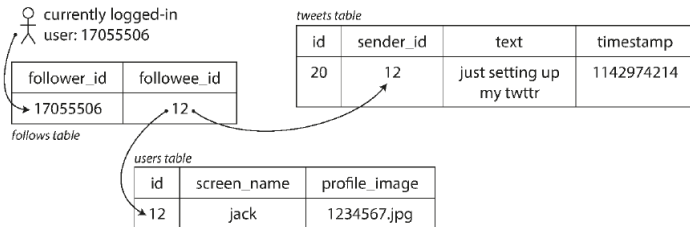
Now there are generally 2 **options** to cope with the load:

1. **Data Loading On Request:** Posting a tweet simply inserts the new tweet into a global collection of tweets. When a user requests their home timeline, look up all the people they follow, find all the tweets for each of those users, and merge them (sorted by time)
2. **Data Pre-Loading (caching):** When a user posts a tweet, look up all the people who follow that user, and insert the new tweet into each of their home timeline caches.



Load of a System: Example (Cont.2)

Option 1: Data Loading On Request: The relational model may look like the following:



In RDBMS the request could an SQL like this:

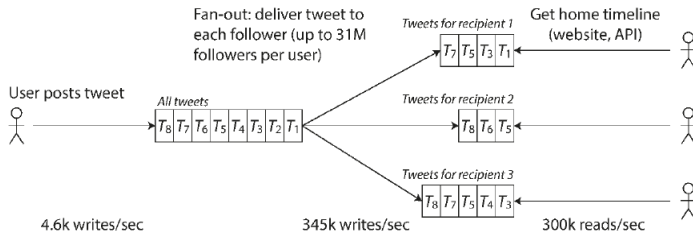
```
1 SELECT tweets.*, users.* FROM tweets
2 JOIN users ON tweets.sender_id
3 = users.id
4 JOIN follows ON follows.followee_id = users.id
5 WHERE follows.follower_id = current_user
```

But for extreme cases the SQL may be too expensive.



Load of a System: Example (Cont.2)

Option 2: Data Pre-Loading (caching):



- The request to read the home timeline is then cheap, because its result has been computed ahead of time.

Comparison Between Options

- The first version of Twitter used approach 1, but the systems struggled to keep up with the load of home timeline queries, so the company switched to approach 2.
- However, the downside of approach 2 is that posting a tweet now requires a lot of extra work.
 - ▶ On average, a tweet is delivered to about 75 followers, so 4.6k tweets per second become 345k writes per second to the home timeline caches.
 - ▶ But number of followers per user **varies wildly i.e. from 30 to 30 million** This means that a single tweet may result in over 30 million writes to home timelines! Doing this in a timely manner is a huge challenge.
- Amazingly, in reality **both options are used** based on scenario. For low to less number of followers tweets Option 1 works fine. And for tweets from persons with millions of followers will be treated by Option 2.

In the example of Twitter, the **distribution of followers per user** is a key load parameter for discussing scalability, since it determines the fan-out load. Your application may have very different characteristics, but you can apply similar principles to reasoning about its load.



Performance of a Data System

Once you have described the load on your system, you can investigate what happens when the load increases. You can look at it in **two ways**:

- i. When you **increase a load parameter** and **keep the system resources (CPU, memory, network bandwidth, etc.) unchanged**, how is the **performance** of your system affected?
- ii. When you **increase a load parameter**, how much do you need to increase the resources if you want to keep **performance** unchanged?

In both cases the key area is **performance of a system**. Lets explore it.



Performance of a Data System

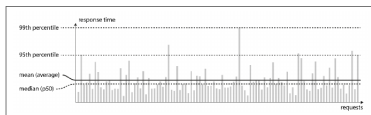
- In a batch processing system such as Hadoop, we usually care about **throughput**: *the number of records we can process per second, or the total time it takes to run a job on a dataset of a certain size*
- In online systems, what's usually more important is the service's **response time** *that is, the time between a client sending a request and receiving a response.*

The response time for a system normally varies a lot. Hence the **distribution of values of response time** is very important to consider.



Performance of a Data System: Response Time

- Its common to see the **average response** time of a service reported. But doesnt tell you how many users actually experienced that delay.
- Usually it is **better** to use **percentiles**. Median is the halfway point, it gives better idea of the system performance. **Example:** Median response time is 150 ms, that means half all requests return in less than 150 ms, and half your requests take longer than that. The median is also known as the **50th percentile**, and sometimes abbreviated as p50.



- **High percentiles of response times**, also known as tail latencies, are **important** because they directly affect users experience of the service.
Example: Amazon describes response time requirements for internal services **in terms of the 99.9th percentile**, even though it only affects 1 in 1,000 requests.

This is because the customers with the slowest requests are often those who have the most data on their accounts because they have made many purchases – that is, theyre the most valuable customers.

Relational Databases: Limitations

- Relational databases have been the dominant type of database used for database applications **for decades**.

Source: Chapter 1, NoSQL for Mere Mortals by Dan Sullivan



Relational Databases: Limitations

- Relational databases have been the dominant type of database used for database applications **for decades**.
- Relational databases **addressed many of the limitations** of flat filebased data stores, hierarchical databases, and network databases.

Source: Chapter 1, NoSQL for Mere Mortals by Dan Sullivan



Relational Databases: Limitations

- Relational databases have been the dominant type of database used for database applications **for decades**.
- Relational databases **addressed many of the limitations** of flat filebased data stores, hierarchical databases, and network databases.
- However, due to **internet** the nature of business applications has changed significantly. And Relational DB losses its suitability in many cases.

Source: Chapter 1, NoSQL for Mere Mortals by Dan Sullivan



Relational Databases: Limitations

- Relational databases have been the dominant type of database used for database applications **for decades**.
- Relational databases **addressed many of the limitations** of flat filebased data stores, hierarchical databases, and network databases.
- However, due to **internet** the nature of business applications has changed significantly. And Relational DB losses its suitability in many cases.
- Companies such as Google, LinkedIn and Amazon found that **supporting large numbers of users** on the Web was **different** from supporting much smaller numbers of business users.

Source: Chapter 1, NoSQL for Mere Mortals by Dan Sullivan



Relations Database: Limitations (Cont.)

Today, Web Application deals with large volumes of data and extremely large numbers of users.

- Large volumes of read and write operations



Relations Database: Limitations (Cont.)

Today, Web Application deals with large volumes of data and extremely large numbers of users.

- Large volumes of read and write operations
- Low latency response times



Relations Database: Limitations (Cont.)

Today, Web Application deals with large volumes of data and extremely large numbers of users.

- Large volumes of read and write operations
- Low latency response times
- High availability



No SQL: Why?

Web applications serving **millions of users** or more are difficult to implement with relational databases.

4 desired characteristics for large-scale data management:

1. Scalability
2. Cost
3. Flexibility
4. Availability



No SQL: Scalability

Scalability is the ability to efficiently meet the needs for varying workloads.

2 options are there:

1. **Scale up.** Database administrators could choose to scale up, which is **upgrading an existing database server to add additional** processors, memory, network bandwidth, or other resources that would improve performance on a database management system or replacing an existing server with one with more CPUs, memory, and so on.



No SQL: Scalability

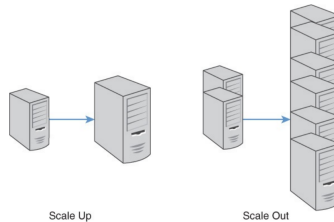
Scalability is the ability to efficiently meet the needs for varying workloads.

2 options are there:

1. **Scale up.** Database administrators could choose to scale up, which is **upgrading an existing database server to add additional** processors, memory, network bandwidth, or other resources that would improve performance on a database management system or replacing an existing server with one with more CPUs, memory, and so on.
2. **Scale out.** If there is a spike in traffic to a website, **additional servers can be brought online** to handle the additional load. When the spike subsides and traffic returns to normal, some of those additional servers can be shut down. Adding servers as needed is called scaling out.



No SQL: Scale up and Scale out



- Scale up in Relational DB is possible, but incurs cost.
- Scale out in Relational DB is hard to ensure, because Transaction (i.e. ACID) management in multiple servers is challenging.
- Scaling out is more flexible than scaling up. Servers can be added or removed as needed when scaling up.
- NoSQL databases are designed to utilize servers available in a cluster with minimal intervention by database administrators.

No SQL: Cost

The cost of database licenses is an obvious consideration for any business or organization.

- **Proprietary Database:** Web applications may have spikes in demand that increase the number of users utilizing a database at any time. Should users of the RDBMS pay for the number of peak users or the number of average users? How should they budget for RDBMS licenses?
- **Open Source Database:** The software is free to use on as many servers of whatever size needed because open source developers do not typically charge fees to run their software. Fortunately for NoSQL database users, the major NoSQL databases are available as open source.



No SQL: Flexibility

- Relational Database has the flexibility in its design phase (i.e. suitable for a wide array of applications). But it **lacks flexibility** in many cases as well.
- **For instance:** An e-commerce application where products are sold. Each product has different set of attributes. **For example**, PC has its CPU, RAM, Display while a Photocopier has its size, capacity, speed of copying etc.
 - ▶ A database designer could create separate tables for each type of product or define a table with as many different product attributes as she could imagine at the time she designs the database.
 - ▶ Unlike relational databases, some NoSQL databases do not require a fixed table structure.



No SQL: Availability

- Today we need systems/applications live for 24x7. If your favorite social media or e-commerce site were frequently down when you tried to use it, you would likely start looking for a new favorite.
- NoSQL databases are designed to take advantage of multiple, low-cost servers. When one server fails or is taken out of service for maintenance, the other servers in the cluster can take on the entire workload.



Distributed Databases

- Recall the motivations for NoSQL, including the need for **scalability, flexibility, cost control, and availability**.

Source: Chapter 2, NoSQL for Mere Mortals by Dan Sullivan



Distributed Databases

- Recall the motivations for NoSQL, including the need for **scalability, flexibility, cost control, and availability**.
- A common way to meet these needs is by designing data management systems to work across **multiple servers**, that is, as a distributed system.

Source: Chapter 2, NoSQL for Mere Mortals by Dan Sullivan



Distributed Databases

- Recall the motivations for NoSQL, including the need for **scalability, flexibility, cost control, and availability**.
- A common way to meet these needs is by designing data management systems to work across **multiple servers**, that is, as a distributed system.
- NoSQL offers some level of **operational simplicity**

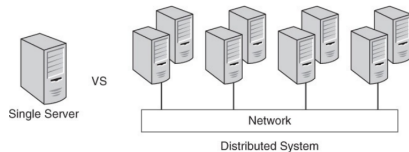
You can add and remove servers as needed rather than adding or removing memory, CPUs, and so on from a single server. Also, some NoSQL databases include features that automatically detect when a server is added or removed from a cluster.

Source: Chapter 2, NoSQL for Mere Mortals by Dan Sullivan



Objectives of Distributed Databases

NoSQL often deploy Distributed Database Architecture.



The database management systems must do three things:

- i. Store data persistently
- ii. Maintain data consistency
- iii. Ensure data availability

Store Data Persistently

- Data must be stored persistently; that is, it must be stored in a way that data is **not lost** when the database **server is shut down**.
- If data were only stored in memory that is, RAM then it would be lost when power to the memory is lost. Only data that is stored on disk, flash, tape, or other **long-term storage** is considered persistently stored.
- Although read/write operation in long-term storage is much slower in comparison to RAM, the response time is minimized using a number of techniques such as **Indexing**.

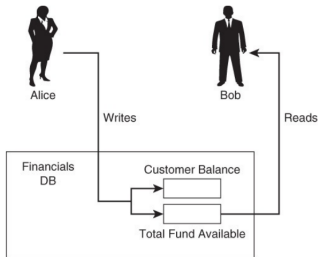


Maintain Data Consistency

- Data **must not be contradictory** as per the business logic.
- Data may inconsistent due to **hardware failure**, but it is very **rare!!**
- **Common cause** of data inconsistency is data read/write operation by **multiple users at the same time**.



Data Consistency: Example



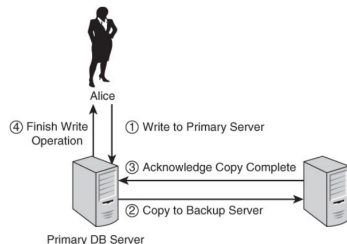
- Consider a small business with two partners, Alice and Bob. Alice is using a database application to update the company's financial records.
- Bob is checking the total fund available **at the same time**.
What balance will Bob see?

Ensure Data Availability

- Data should be available whenever it is needed. This is difficult to guarantee.
- A database that runs on a **single server** can be unavailable for a large number of reasons.
- One way to avoid the problem of an unavailable database server is to have **two** database servers: **primary server** and **back-up server**
- 2 server setup is controlled by **two-phase commit** protocol.



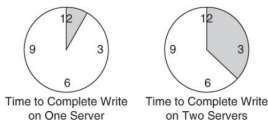
Two-phase Commit



- When the database is in use, any changes to the primary database are reflected in the backup database as well.
- With data consistent on two database servers, you can be sure that if the primary database fails, you can switch to using the backup database and know that you have the same data on both.

Two-phase Commit (Cont.)

- The **advantage** of using two database servers is that it enables the database to remain available even if one of the servers fails.
- But it **incurs cost**. A write operation is not complete until both databases are updated successfully. And it incurs additional time due to amount of data written, the speed of the disks, the speed of the network between the two servers.



Availability and Consistency: Application Diversity

- When two database servers must keep consistent copies of data, they **incur longer times** to complete a transaction.
- This is acceptable in applications that require both consistency and high availability at all times. **Financial systems at a bank**, for example, fall into this category.
- There are applications, however, in which the **fast database operations are more important than maintaining consistency** at all times.
For example, an **e-commerce site** might want to maintain copies of your shopping cart on two different database servers. If one of the servers fails, your cart is still available on the other server.
- How long should the customer wait after clicking on an Add to My Cart button? Ideally, the interface would respond immediately so the customer could keep shopping. For a slow and sluggish system, the **customer might switch to another site**.



e-commerce Site Challenge

- Let the program **write the updates to one database** and then let the program know the data has been saved. The interface can indicate to the customer that the product has been added to the cart.
- While the customer receives the message that the cart has been updated, the database management system is making a copy of the newly updated data and **writing it to another server**.
- There is a **brief period of time** when the customers cart on the **two servers is not consistent**, but the customer is able to continue shopping anyway.
- In this case, we are **willing to tolerate inconsistency for a brief period of time** knowing that eventually the two carts will have the same products in it.



e-commerce Site Challenge (Cont.)

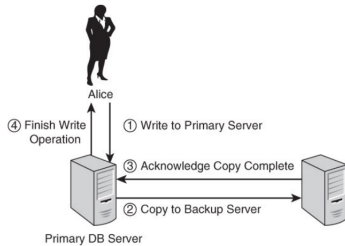


Figure: Strict Consistency: Traditional DBMS

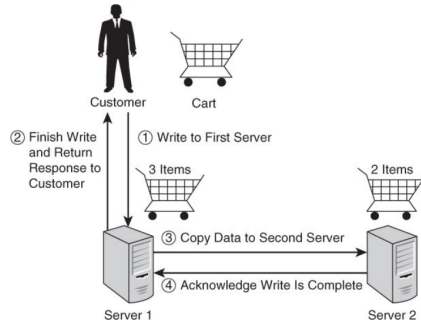


Figure: Liberal Consistency: NoSQL Trend

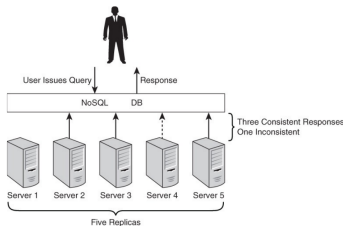
- There is only **a small chance** someone else would read that customers cart data anyway from the server2.

Eventual Consistency

- NoSQL databases often implement **eventual consistency**; that is, there might be a period of time where copies of data have different values, but eventually all copies will have the same value.
- This raises the possibility of a user **querying the database and getting different results** from different servers in a cluster.
- NoSQL databases often use the concept of **quorums** when working with reads and writes.



Eventual Consistency: Quorums



- A quorum is the number of servers that must respond to a read or write operation for the operation to be considered complete.
- When a read is performed, the NoSQL database **reads data from, potentially, multiple servers.**
- **For example,** assume data in a NoSQL database is replicated to five servers and you have set the read threshold to 3. As soon as **three servers respond with the same response**, the result is returned to the user.
- You can **vary the threshold** to improve response time or consistency.

The CAP Theorem

The CAP Theorem

The CAP theorem, also known as Brewers theorem after the computer scientist who introduced it, states that distributed databases cannot have consistency (C), availability (A), and partition protection (P) all at the same time.

✓ This means that when a network partition occurs, a database **must choose to sacrifice either consistency or availability.**



The CAP Theorem (Cont.)

CAP Theorem: 3 Guarantees–

- i. **Consistency (C):** In a consistent system, every client receives the same, most up-to-date data at the same time, regardless of which node they connect to. After a write operation, all subsequent reads across all nodes must return the newly written value.
- ii. **Availability (A):** An available system guarantees that every request to a non-failing node receives a response. The system remains operational and responsive even if some nodes have failed. It doesn't guarantee the data is the most recent.
- iii. **Partition Tolerance (P):** A partition-tolerant system continues to operate despite network partitions. A network partition is a break in communication between nodes, where some nodes can't communicate with others. Because network failures are inevitable in distributed systems, partition tolerance is a mandatory characteristic for any modern distributed database.

Source: Google Gemini and relevant video by ByteByteGo



The CAP Theorem (Cont.2)

CAP Theory for NoSQL: Because network partitions are a given in any distributed system, the CAP theorem effectively forces a choice between consistency and availability. Hence, NoSQL databases are often categorized either of the:

- i. **CP (Consistency and Partition Tolerance)**: These systems prioritize consistency. When a partition occurs, they will refuse to serve requests on the side of the partition that doesn't have the most current data, thus sacrificing availability to ensure correctness. **Example**: Redis and HBase.
- ii. **AP (Availability and Partition Tolerance)**: These systems prioritize availability. When a partition occurs, they will continue to serve requests on all nodes, even if it means some nodes might return outdated or "stale" data. The data will eventually become consistent once the network partition is resolved. This is known as eventual consistency. **Example**: Cassandra and CouchDB



ACID and BASE

- ACID is an acronym derived from four properties implemented in relational database management systems.
- BASE is an acronym for properties common to NoSQL databases.



ACID Properties

- **A** is for atomicity. Atomicity, as the name implies, describes a unit that cannot be further divided.
- **C** is for consistency. In relational databases, this is known as **strict consistency**.
- **I** is for isolation. Isolated transactions are not visible to other users until transactions are complete.
- **D** is for durability. This means that once a transaction or operation is completed, it will remain even in the event of a power loss.

Relational database management systems are designed to support ACID transactions.



BASE Properties

- **BA** is for basically available. This means that there can be a partial failure in some parts of the distributed system and the rest of the system continues to function.

For example, if a NoSQL database is running on 10 servers without replicating data and one of the servers fails, then 10% of the users queries would fail, but 90% would succeed.

- **S** is for soft state. Usually in computer science, the term soft state means data will expire if it is not refreshed.

Here, in NoSQL operations, it refers to the fact that data may eventually be overwritten with more recent data. This property overlaps with the third property of BASE transactions, eventually consistent.

- **E** is for eventually consistent. This means that there may be times when the database is in an inconsistent state.



Types of Eventual Consistency

Eventual consistency is such an important aspect of NoSQL databases. There are several types of eventual consistency:

1. Casual consistency
2. Read-your-writes consistency
3. Session consistency
4. Monotonic read consistency
5. Monotonic write consistency



Casual Consistency

Casual consistency ensures that the database **reflects the order** in which operations were updated.

Example: if Alice changes a customers outstanding balance to \$1,000 and one minute later Bob changes it to \$2,000, all copies of the customers outstanding balance will be updated to \$1,000 before they are updated to \$2,000.



Read-Your-Writes Consistency

Read-your-writes consistency means that once you have updated a record, all of your reads of that record will **return the updated value**. You would **never retrieve a value inconsistent** with the value you had written.

Example: Alice updates a customers outstanding balance to \$1,500. The update is written to one server and the replication process begins updating other copies. During the replication process, Alice queries the customers balance. She is guaranteed to see \$1,500 when the database supports read-your-writes consistency.



Session Consistency

Session consistency ensures read-your-writes **consistency during a session**.

- ✓ **As long as the conversation (session) continues**, the database remembers all writes you have done during the conversation. **If the session ends and you start another session** with the same server, there is **no guarantee it will remember the writes you made in the previous session**.
- ✓ A **session may end** if you log off an application using the database or if you do not issue commands to the database for so long that the database assumes you no longer need the session and abandons it.



Monotonic Read Consistency

Monotonic read consistency ensures that if you issue a query and see a result, you will **never see an earlier version of the value**.

Example: Alice is yet again updating a customer's outstanding balance. The outstanding balance is currently \$1,500. She updates it to \$2,500. Bob queries the database for the customer's balance and sees that it is \$2,500. If Bob issues the query again, he will see the balance is \$2,500 even if all the servers with copies of that customer's outstanding balance have not updated to the latest value.



Monotonic Write Consistency

Monotonic write consistency ensures that if you were to issue several update commands, they would be **executed in the order you issued them.**



Concept of Aggregates Data Model¹

- The dominant data model of the last couple of decades is the relational data model, which is best visualized as a set of tables.
- Each NoSQL solution has a different model that it uses, which we put into four categories widely used in the NoSQL ecosystem: key-value, document, column-family, and graph.
- Of these, the first three share a common characteristic of their data models which we will call aggregate orientation.



Aggregates

- Aggregate is a term that comes from Domain-Driven Design [Evans]. In Domain-Driven Design, an aggregate is a collection of related objects that we wish to treat as a unit.
- Typically, we like to update aggregates with atomic operations and communicate with our data storage in terms of aggregates.
- **What you want** must match with **What you store**.
- Aggregate can work like a Node which is independent of other nodes.
- It **incurs redundancy** but **scales-up more efficiently**.
- **Aggregate boundary** must be carefully designed.



Aggregates: Example

<http://www.linkedin.com/in/williamhgates>



Bill Gates

Greater Seattle Area | Philanthropy

Summary

Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader. Avid traveler. Active blogger.

Experience

Co-chair • Bill & Melinda Gates Foundation
2000 – Present

Co-founder, Chairman • Microsoft
1975 – Present

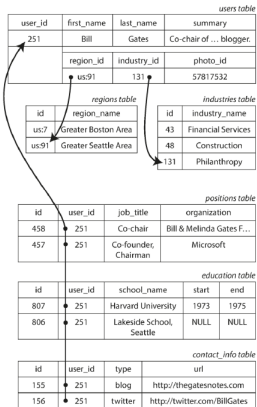
Education

Harvard University
1973 – 1975

Lakeside School, Seattle

Contact Info

Blog: thegatesnotes.com
Twitter: @BillGates



Aggregates: Example (Cont.)

<http://www.linkedin.com/in/williamhgates>



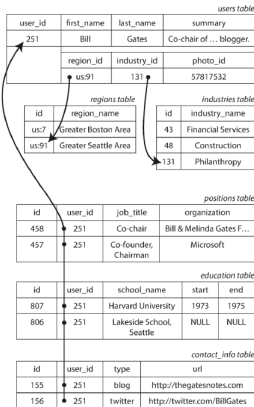
Bill Gates
Greater Seattle Area | Philanthropy

Summary
Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader. Avid traveler. Active blogger.

Experience
Co-chair • Bill & Melinda Gates Foundation
2000 – Present
Co-founder, Chairman • Microsoft
1975 – Present

Education
Harvard University
1973 – 1975
Lakeside School, Seattle

Contact Info
Blog: thegatesnotes.com
Twitter: @BillGates



There is a one-to-many relationship from the user to these items, which can be represented in various ways:

- **Option 1:** the most common normalized representation is to put positions, education, and contact information in separate tables, with a foreign key reference to the users table, as in the Figure.
- **Option 2:** Later versions of the SQL standard added support for structured datatypes and XML data; this allowed multi-valued data to be stored within a single row, supported by Oracle, IBM DB2, MS SQL Server, PostgreSQL.
- **Option 3:** To encode jobs, education, and contact info as a JSON or XML document, store it on a text column in the database, and let the application interpret its structure and content.

Aggregates: Example (Cont.2)

<http://www.linkedin.com/in/williamhgates>



Bill Gates
Greater Seattle Area | Philanthropy

Summary

Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader.avid traveler. Active blogger.

Experience

Co-chair - Bill & Melinda Gates Foundation
2006 - Present
Co-founder, Chairman - Microsoft
1975 - Present

Education

Harvard University
1973 - 1975
Lakeside School, Seattle

Contact Info

Blog: thegatesnotes.com
Twitter: willGates

user_id	first_name	last_name	summary
251	Bill	Gates	Co-chair of ... Blogger

id	region_name	id	industry_name
us07	Greater Boston Area	43	Financial Services
us01	Greater Seattle Area	48	Construction
		131	Philanthropy

id	user_id	job_title	organization
458	251	Co-chair	Bill & Melinda Gates F...
457	251	Co-founder, Chairman	Microsoft

id	user_id	school_name	start	end
807	251	Harvard University	1973	1975
806	251	Lakeside School, Seattle	NULL	NULL

id	user_id	type	url
155	251	blog	http://thegatesnotes.com
156	251	twitter	http://twitter.com/WillGates

There is a one-to-many relationship from the user to these items, which can be represented in various ways:

- **Option 3:** To encode jobs, education, and contact info as a JSON or XML document...
- For a data structure like a résumé, which is mostly a self-contained document, a JSON representation can be quite appropriate. Document-oriented databases like MongoDB, RethinkDB, CouchDB, and Espresso support this data model.
JSON is :

- ▶ Code is more Readable
- ▶ Smaller data size
- ▶ Faster in parsing speed
- ▶ Supports various data types

Aggregates: Sample JSON document

```
1      {
2          "user_id":
3              251,
4          "first_name": "Bill",
5          "last_name":
6              "Gates",
7          "summary":
8              "Co-chair of the Bill & Melinda Gates... Active blogger.",
9          "region_id":
10             "us:91",
11          "industry_id": 131,
12          "photo_url":
13              "/p/7/000/253/05b/308dd6e.jpg",
14
15          "positions": [
16              {"job_title": "Co-chair", "organization": "Bill & Melinda Gates
17                  Foundation"},
18              {"job_title": "Co-founder, Chairman", "organization": "Microsoft"}
19          ],
```



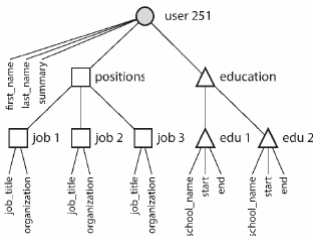
Aggregates: Sample JSON document

```
1
2
3  "education": [
4    {"school_name": "Harvard University",
5      "start": 1973, "end": 1975},
6    {"school_name": "Lakeside School, Seattle", "start": null, "end": null}
7  ],
8  "contact_info": {
9    "blog":
10     "http://thegatesnotes.com",
11     "twitter": "http://twitter.com/BillGates"
12  }
13 } // end of object
```



Aggregates: JSON Example

- The JSON representation has better locality than the multi-table schema.
- If you want to fetch a profile in the relational example, you need a number of joins.
- In the JSON representation, on the other hand, all the relevant information is in one place, and one query is sufficient.
- JSON representation makes a single tree structure to capture one to many relationships.



Aggregates: Boundary

- An **aggregate** is a collection of objects or data that are treated as a single, logical unit. For example, in an e-commerce application, a "Customer" aggregate might include not just the customer's personal details but also their address and a list of their recent orders. All of this information is considered part of the same aggregate because it's frequently accessed and modified together.
- The **boundary** is the line that separates an aggregate from other data in the database. Its primary purpose is to enforce atomicity. This means that any changes or updates made to the aggregate's data must happen all at once. The entire aggregate is either successfully updated, or the transaction fails completely, leaving the data in its original state. This is different from the relational database model, where you might need to perform a transaction across multiple tables, which can be slower and more complex.



Aggregates: More

Consequences of Aggregate Orientation:

- Aggregation is not a logical data property It is all about how the data is being used by applications.
- An aggregate structure may be an obstacle for others but help with some data interactions.
- It has an important consequence for transactions. NoSQL databases dont support ACID transactions thus sacrificing consistency.

Strength of Aggregates:

- It can be used as a primary data source for online applications.
- It provides fast performance and horizontal Scalability.

Weakness of Aggregates:

- No standard rules. (highly personalized issue)
- Limited query capabilities.
- Doesn't work well with relational data. Hence, not so popular in the enterprise area.
- Extra effort needed by the UI designers.



Types of NoSQL Databases

The most widely used types of NoSQL databases are:

- Key-value pair databases **e.g. Riak KV (Key-Value)**
- Document databases **e.g. MongoDB, Apache CouchDB**
- Column family store databases **Apache Cassandra**
- Graph databases **e.g. Neo4j** ✓

Note: NoSQL databases do not have to be implemented as distributed systems. Many can **run on a single server**. Some of the most interesting and appealing features of NoSQL databases, however, **require a distributed implementation**.



Key-value Databases: Basic characteristics

- The simplest NoSQL data store:
 - ✓ A hash table (map)
 - ✓ When all access to the database is via primary key
- Like a table in RDBMS with two columns:
 - ✓ ID = key
 - ✓ NAME = value (any type)
- Basic operations:
 - ✓ **get** the value for the key
 - ✓ **put** a value for a key (existing value will be overwritten)
 - ✓ **delete** a key from the data store



Key-value Databases: Examples

Representatives



MemcachedDB



ORACLE®
BERKELEY DB

Hamster DB
embedded database



not open-source

open-source
version



**Project
Voldemort**

Key-value Databases: Suitable Applications

- **Storing Session Information**

- ✓ Every web session is assigned a unique `session_id` value
- ✓ Everything about the session can be stored by a single **PUT** request or retrieved using a single **GET**

- **User Profiles, Preferences**

Every user has a unique `user_id`, `user_name` + preferences (e.g., language, colour, time zone, which products the user has access to, ...)

- **Shopping Cart Data**



Key-value Databases: When **NOT** To Use

- Relationships among Data

- ✓ Relationships between different sets of data

- Multioperation Transactions

- ✓ Saving multiple keys. Failure to save any one of them → **revert or roll back** the rest of the operations. (Since it does not ensure ACID)

- Query by Data

- ✓ **Search** the keys based on something found in the **value part**

- Operations by Sets

- ✓ **No way to operate** upon **multiple keys** at the **same time**



Key-value Pair Databases

These databases are modeled on two components: **keys** and **values**.

Keys:

- Keys are identifiers associated with values. They are analogous to tags you get when you check luggage at the airport.
- **Example:** a key-generating program for an **e-commerce website**. Need to track **five pieces of information** as follows:
 - i. customers account number
 - ii. name
 - iii. address
 - iv. number of items in the shopping cart
 - v. customer type indicator.



Key-value Pair Databases: Key (Cont.1)

- All of these values are associated with a customer, so you can generate a sequential number for each customer.

For example, data about the first customer in the system would use keys **1.accountNumber**, **1.name**, **1.address**, **1.numItems**, and **1.custType**

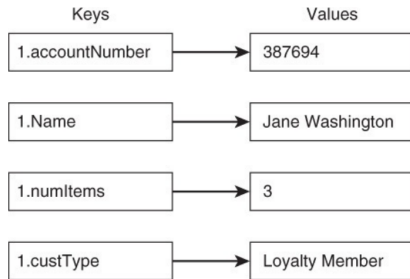


Figure: Two-part data structure: Key and its value

Key-value Pair Databases: Key (Cont.2)

- Multiple entities involving similar attribute will **cause a problem**, since in this case one key will point to two different values at two different times. But at the end, only one value will persist (which has been written most recently)

For instance: 1.Name in another entity means Name of the first branch of the company. But if you save 1.Name value as the branch name and query next time to find the name of the 1st customer by the key 1.Name, it will generate wrong answer (i.e. branch name)

- Solution:** Use a key-naming convention that includes the entity type.

For Instance: the prefix **cust** and **brn** could be used to differentiate them:

cust1.name and brn1.name are now two different keys



Key-value Pair Databases: Key (Cont.3)

- To differentiate among entities, data structures for separate collections of identifiers within a database are used, commonly called **buckets**

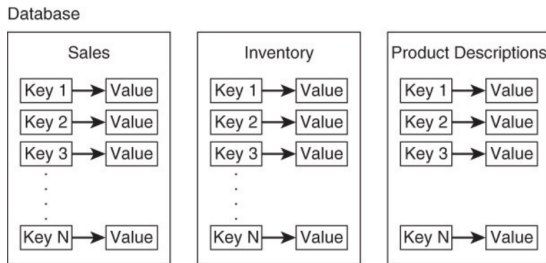


Figure: Separate namespaces within a single database

Key-value Pair Databases: Key (Cont.3)

Buckets Example Code: Lets store an object containing information about a dog named Rufus. Well store that object in the **key=rufus** in the bucket dogs, which bears the animals bucket type.

Notice that we specified both a **value for the object, i.e. WOOF!**, and a content type, text/plain

Python Code for **Riak KV**:

```
1 | bucket = client.bucket_type('animals').bucket('dogs')
2 | obj = RiakObject(client, bucket, 'rufus')
3 | obj.content_type = 'text/plain'
4 | obj.data = 'WOOF!'
5 | obj.store()
```



Key-value Pair Databases: Value

- Values can be as **simple** as a string or **complex** values, such as images or binary objects, too.
- It offers **great flexibility** when storing values.
 - ✓ For example, its string length may vary.
 - ✓ Another example: An employee database might include photos of employees using keys such as Emp328.photo. That key could have a picture stored as a binary large object (BLOB) type or a string value such as Not available.
- Key-value databases typically **do not enforce checks on data types of values**. Hence, software developers must implement checks in their programs.



Key-Value and Relational Databases: Difference

- Key-value databases are modeled on minimal principles for storing and retrieving data.
- Unlike in relational databases, there are no tables, so there are no features associated with tables, such as columns and constraints on columns.
- There is no need for joins in key-value databases, so there are no foreign keys. Key-value databases do not support a rich query language such as SQL.



Key-Value and Relational Databases: Difference (Cont.)

- But, there are parallels between the key-naming convention and tables, primary keys, and columns :

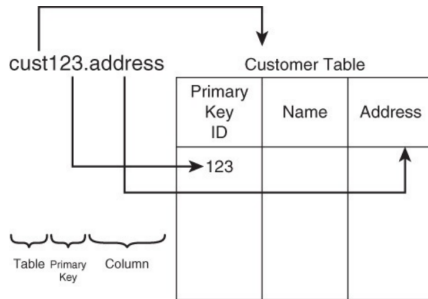


Figure: Mapping from KV to Relational DB

Graph Databases

Graph databases are the most specialized of the four NoSQL databases discussed in this book. Instead of modeling data using columns and rows, a graph database uses structures called **nodes and relationships (edges)**. It is based on **Principles of Graph Theory**.



Relational Databases: Major Weakness

- Since the 80s, they have been the power-horse of most software applications and continue to be so today. Relational databases (RDBMSs) were initially designed to codify paper forms and tabular structures, and they do that exceedingly well.
- RDBMSs **don't** robustly store **relationships between data elements**. They are not designed to capture this rich relationship information.
- Relational database heavily depends on **JOIN operations**. These operations are **compute and memory-intensive and have an exponential cost** as queries grow.



Relational Databases: SQL Strain

Any attempt to solve a connected data problem with a relational database may cause the following performance problems commonly known as **SQL Strain**:

- **A Large Number of JOINS.** When you utilize queries that JOIN many different tables, there's an explosion of complexity and computing resource consumption. This results in a corresponding increase in query response times.
- **Numerous Self-JOINS (or Recursive JOINS).** Self-JOIN statements are common for hierarchy and tree representations of data. It also incurs higher response time.
- **Frequent Schema Changes.** Relational databases aren't designed for frequent modifications of its structure. But today's agile nature of applications demand such frequent changes. A more flexible data model is needed.



Graph Database an Alternative of Relational Databases

- For highly structured, predetermined schemas, an RDBMS is the perfect tool. But for ever-changing nature of applications we need an efficient alternative – Graph is a good option in this regard.
- Graph database offers better performance and flexibility (and agility).
- In fact, the flexibility of a graph model **allows you to add new nodes and relationships without compromising your existing network** or expensively migrating your data. All of your original data (and its original relationships) remain intact.



Graph Database an Alternative of Relational Databases (Cont.)

- With data relationships at their center, graphs are **incredibly efficient** when it comes to query speeds, even for deep and complex queries. In the book [Neo4j in Action](#), the authors performed an experiment between a relational database and a Neo4j graph database.
- ✓ Their experiment used a basic social network to find friends-of-friends connections to a depth of five degrees. Their dataset included 1,000,000 people each with approximately 50 friends. Result is given as follows:

Depth	RDBMS execution time(s)	Neo4j execution time(s)	Records returned
2	0.016	0.01	~2,500
3	30.267	0.168	~110,000
4	1543.505	1.359	~600,000
5	Unfinished	2.132	~800,000

Graph Database an Alternative of Relational Databases: Example

Consider the following sample relational database:

User					
UserID	User	Address	Phone	Email	Alternate
1	Alice	123 Foo St.	12345678	alice@example.org	alice@neo4j.org
2	Bob	456 Bar Ave.		bob@example.org	
...	...	...	...	...	...
99	Zach	99 South St.		zach@example.org	

Order	
OrderID	UserID
1234	1
5678	1
...	...
5588	99

LineItem		
OrderID	ProductID	Quantity
1234	765	2
1234	987	1
...	...	...
5588	765	1

Product		
ProductID	Description	Handling
321	strawberry ice cream	freezer
765	potatoes	
...	...	
987	dried spaghetti	

- Q1: What items did a customer buy?
- Q2: Which customers bought this product?

Both queries will be very slow because of several JOINS.

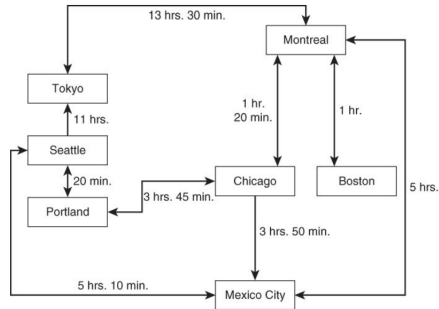


Graph Databases: Motivating Example

- **Example:** A node could be a city, and a relationship between cities could be used to store information about the distance and travel time between cities.

Objective: What is the best route from Seattle to Boston?

- **Solution:** Solution to the Shortest Path Problem



Graph Databases: Motivation (Cont.)

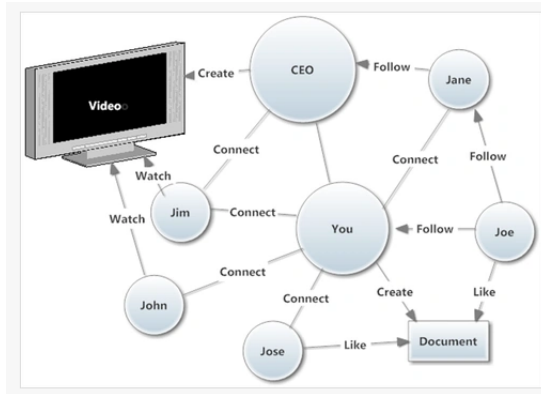
City 1	City 2	Flying Time
Montreal	Boston	1 hr.
Montreal	Chicago	1 hr. 20 min.
Montreal	Tokyo	13 hr. 30 min.
Montreal	Mexico City	5 hr.
Chicago	Mexico City	3 hr. 50 min.
Chicago	Portland	3 hr. 45 min.
Portland	Seattle	20 min.
Seattle	Tokyo	11 hr.
Seattle	Mexico City	5 hr. 10 min.

- In RDBMS it is **hard to achieve**. Performing this same kind of analysis in a relational database would be more challenging.
- Querying is more difficult. You would have to write multiple SQL statements or use **specialized recursive statements** if they are provided **to find paths** using the table representation of the data.



Graph Databases: Motivation (Cont.)

- Social Network with different relationships.



Graph Databases: Motivation Summary

- Graph Database offers flexible and efficient (i.e. scalable) data operation specially with high degree of data dependency (of large volume)
- Graph Database applications include: social network modeling, e-mail fraud detection, Recommendations in Real Time and so on.
- Basic operations of Graph (such as search, find the degree, find sub-graph) can be used to satisfy user Query in these applications.



Examples of Graph Databases¹



Example of Graph Databases: neo4j

Neo4j is one of the worlds leading graph databases. It is a **highly scalable NoSQL open-source database** developed using **Java**.

Main Features¹:

- **Property graph data model.** Enables intuitive and flexible data modeling, easy to capture complex data relationships.
- **Native graph processing and storage.** Optimizes data retrieval and graph traversals, ensuring swift and efficient handling of large datasets and complex queries.
- **ACID compliant transactions.** Guarantees reliable data processing, maintaining data accuracy and trustworthiness across all transactions.



Example of Graph Databases: neo4j

Main Features¹ (Cont.):

- **Cypher graph query language.** Provides a powerful yet user-friendly method for querying graph data, simplifying the extraction of meaningful insights from interconnected data.
- **High-performance native API.** Ensures efficient interaction with the database, crucial for applications requiring low-latency and high-throughput database interactions.
- **Cypher client.** Facilitates seamless execution of Cypher queries from applications, enhancing dynamic and interactive user experiences.
- **Driver support for multiple programming languages.** Offers flexibility in development by providing drivers for various programming languages, including **C#, Go, Java, JavaScript, and Python**, ensuring easy integration into diverse technology stacks.



Neo4j : Basics of Cypher

- Cypher statements to create vertices¹:

```
1 CREATE (robert:Developer { name: 'Robert Smith' })
2 CREATE (andrea:Developer { name: 'Andrea Wilson' })
3 CREATE (charles:Developer { name: 'Charles Vita' })
```

- ✓ These three statements create three vertices.
- ✓ The text `robert: Developer` creates a developer vertex with a label of robert.
- ✓ The text `{ name: 'Robert Smith' }` adds a property to the node to store the name of the developer.

Neo4j : Basics of Cypher (Cont.)

- Edges are added with create statements as well¹. For example:

```
1 | CREATE (robert)-[FOLLOWS]->(andrea)
2 | CREATE (andrea)-[FOLLOWS]->(charles)
```

Neo4j : Gremlin

- Gremlin: It is a query language used to retrieve data from and modify data in the graph. (part of Apache TinkerPop Framework founded and maintained by Apache)
- It provides both Depth-First and Breadth-First Searches for a given graph.
- Interested readers may consult the following urls:

i. <https://kelvinlawrence.net/book/Gremlin-Graph-Guide.html#tpintro>

ii. [https:](https://tinkerpop.apache.org/docs/current/reference/#_tinkerpop_documentation)

[//tinkerpop.apache.org/docs/current/reference/#_tinkerpop_documentation](https://tinkerpop.apache.org/docs/current/reference/#_tinkerpop_documentation)



Thank You!!!

